Comparing the Efficacy of Neural Networks to Regression Algorithms for use on Numerical and Image Processing

Adam Grant, Alvi Dervishi, John Ray, Luke Sanderson, Samir Rashid

March 2026

Abstract

Machine-learning algorithms are a widespread method used in many fields to study, analyse, and draw inferences from patterns in data, and are able to learn and adapt without explicit instruction. Whilst each algorithm serves the same purpose, the qualities of each one may make it more suitable for specific areas. The objective of this report is to demonstrate the effectiveness of neural networks and logarithmic, linear, and polynomial regression algorithms in predicting distinct datasets. By comparing the effects of linear maximum likelihood estimation, polynomial regression, and logarithmic regression, as well as the applications of Hyperbolic Tangent, Sigmoid, and Rectified Linear Unit activation functions, we independently derive the optimal configurations of feedforward and convolutional neural networks, testing the networks on large quantities of numerical and image data to obtain highly accurate results. The MNIST and Fashion MNIST datasets, stock market data from Yahoo Finance, and Met Office weather data were selected to demonstrate their efficacy; the inclusion of both numerical and image datasets clearly shows the broad applications of all the algorithms. The results of this project show that neural networks have much higher accuracy in image processing, achieving 97.1% on MNIST and 88.2% on Fashion MNIST, whilst linear regression shows greater applicability in financial and weather data.

Contents

1 Introduction **3**

2 Background Theory **4**
2.1 Linear Regression 4
2.2 Polynomial Regression 5
2.3 Logarithmic Regression 5
2.4 Neural Networks 5

3 Stock Data **7**
3.1 Stock Specific Tailoring 7
3.2 Returns 7
3.3 Look-back Window 8

4 Image Processing **8**
4.1 The MNIST and Fashion MNIST datasets 8
4.2 Image Pre-processing 9
4.3 The Softmax Activation Function 10

5 Stock Data Results **10**
5.1 Neural Networks 10
5.2 Polynomial & Logarithmic Regression 14
5.3 Maximum Likelihood Linear Regression 17

6 Temperature Data Results **20**
6.1 Neural Networks 21
6.2 Linear, Polynomial, and Logarithmic Regression 22

7 MNIST Datasets Results **24**
7.1 Linear Regression 24
7.2 Neural Network 24

8 Comparison **25**

9 Limitations **26**

10 Conclusion **27**

1 Introduction

History

In 1943, neurophysiologist Warren McCulloch and logistican Warren Pitts, wrote the first paper considering a non-learning computational model for neural networks [1] This paved the way for neurophysiologist D.O Hobbs to propose a method of machine-learning based on neural plasticity (that is, the more active a synapse is stimulated, the greater its efficacy becomes). This process, known today as Hebbian learning [2], led to early simulations of artificial neural networks in the 1950s[3][4], until an artificial neural network was first implemented by psychologist Frank Rossenblatt, with funding by the United States Office of Naval Research, in 1958[5].

Rossenblatt's first artificial neural network (ANN) was a simple feedforward network without any adaptive hidden layers. These hidden layers were later adapted by Rossenblatt in 1962[6], but the lack of any sufficient back-propagation functions, or the development of advanced activation functions, limited the creation of a usable deep-learning program at this time.

The first functional deep-learning program was published by Mathematician Alexey Ivakhnenko and his associate V.G. Lapa in 1966 [7], with later developments including the introduction of Stochastic Gradient Descent in 1967 by neuroscientist Shun'ichi Amari[8], and the eventual development of Convolutional Neural Networks (CNNs) by the computer scientist Kunihiko Fukushima in 1979[9].

By 2011, the development of ANNs had reached a critical point - with a neural network created for the first time that outperformed human image recognition[10]. This, paired with similar advancements in the field, contributed to the development of the AI sector as it is today. This sector is set to more than quadruple in value by 2030[11], and has had drastic investment in commerce and research alike.

Motivations

Machine learning, in particular neural networks, provides a powerful tool to learn intricate patterns directly from vairous data. This project will attempt to rigorously evaluate the viability and advantages of neural networks compared to classical regression methods. By analysing stock market return, global temperature rise data, alongside the well-known MNIST image datasets, we aim to explore both financial, temperature, and image-based prediction tasks, highlighting the broad applicability of neural networks. Stock price movements are influenced by complex, nonlinear relationships, including short-term trends, momentum, and microstructural effects: a myriad of factors we will try to capture and tame using a neural network. Comparatively, global temperature rise data is dominated by large fluctuations year-upon-year, due to noise and uncertainty in predictions; with less complex relationships arising in the data.

Understanding these differences is not merely an academic exercise. Improved predictive capabilities in finance could inform investment strategies, risk assessment, and algorithmic trading, a better understanding of global temperature rises could lead to advances in environmental science and climate research, while advances in image recognition directly influence fields such as autonomous vehicles, medical imaging, and handwriting recognition. The combination of financial, temperature, and image datasets, alongside architectural comparisons, makes this investigation novel and broadly relevant, with the potential to inspire further research into optimised machine learning strategies for real-world applications.

2 Background Theory

2.1 Linear Regression

The simplest regression fit to data is via linear regression. While providing a straightforward method for modelling data, it is inherently limited to capturing linear relationships between input and output variables.

$$f(x) = wx + b \tag{1}$$

w and b are scalar quantities that must be optimised to give the best linear fit. w is commonly referred to as the weight(s) of the algorithm and b as the bias of the algorithm.

To find the optimal values of w and b, we aim to minimise the Mean Squared Error for the algorithm (given below as l):

$$l = \frac{1}{N}\sum_{i=1}^{N}[y_i - (wx_i + b)]^2 \tag{2}$$

This is done by simply taking the partial derivatives of l in a process known as "Stochastic Gradient Descent". Alternative loss functions can be used here, depending on the type of neural network used:

$$\frac{\partial l}{\partial w} = -\frac{1}{N}\sum_{i=1}^{N} 2x_i[y_i - (wx_i + b)] \tag{3}$$

$$\frac{\partial l}{\partial b} = -\frac{1}{N}\sum_{i=1}^{N} 2[y_i - (wx_i + b)] \tag{4}$$

This process proceeds over a set of iterations, with the weight(s) and bias updated for each. The speed at which these parameters is updated is defined by an additional parameter called the learning rate (α):

$$w \leftarrow w - \alpha\frac{\partial l}{\partial w} \tag{5}$$

$$b \leftarrow b - \alpha\frac{\partial l}{\partial b} \tag{6}$$

Maximum Likelihood Estimation for Linear Regression

A more rigorous statistical interpretation of linear regression is obtained by viewing the observed target values as random variables rather than purely deterministic outputs. In the stock market component of this project, the quantity of interest is the daily log return, and so the regression model is written in the form:

$$y_t = \beta_0 + \beta_1 x_{1,t} + \beta_2 x_{2,t} + \cdots + \beta_p x_{p,t} + \varepsilon_t, \tag{7}$$

where y_t denotes the return to be predicted, the $x_{j,t}$ are engineered input features, the β_j are coefficients to be learned, and ε_t is an error term. A standard assumption is that the errors are independent and normally distributed with a mean of zero and constant variance:

$$\varepsilon_t \sim \mathcal{N}(0, \sigma^2). \tag{8}$$

Under this assumption, the conditional distribution of y_t given the input vector $\mathbf{x}_t$ is:

$$y_t \mid \mathbf{x}_t \sim \mathcal{N}(\mathbf{x}_t^\top \boldsymbol{\beta}, \sigma^2). \tag{9}$$

Rather than minimising the discrepancy between prediction and observation in an ad-hoc manner, one instead selects the parameter values that make the observed data most probable under the assumed model [15][16]. For a sample of N observations, the likelihood is

$$L(\boldsymbol{\beta}, \sigma^2) = \prod_{t=1}^{N} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_t - \mathbf{x}_t^\top \boldsymbol{\beta})^2}{2\sigma^2}\right). \tag{10}$$

Taking logarithms yields the log-likelihood:

$$\log L(\boldsymbol{\beta}, \sigma^2) = -\frac{N}{2}\log(2\pi) - \frac{N}{2}\log(\sigma^2) - \frac{1}{2\sigma^2}\sum_{t=1}^{N}(y_t - \mathbf{x}_t^\top \boldsymbol{\beta})^2, \tag{11}$$

and maximising this expression is equivalent to minimising the negative log-likelihood. For Gaussian errors, this optimisation leads to the same coefficient estimates as ordinary least squares, but the maximum-likelihood viewpoint is especially valuable because it embeds the model in a fully statistical framework and provides a natural route to likelihood-based evaluation [15][18].

2.2 Polynomial Regression

A more complex fit to data is via polynomial regression. This works by extending linear regression to a higher order, which when applied to a one-dimensional dataset of the form $\{(x_i, y_i)\}_{i=1}^{N}$ can be plotted by the equation below:

$$f(x) = w_N x_i^N + w_{N-1} x_i^{N-1} \ + ... + w_2 x_i^2 + w_1 x_i + b \tag{12}$$

As seen previously for linear regression, the optimal values of w and b can be determined by minimising their values from taking derivatives 3 4.

2.3 Logarithmic Regression

Logarithmic regression is another more complex fitting tool, which takes advantage of the log function as opposed to purely polynomial expressions. This regression takes the form of the equation below:

$$f(x) = wlog|x + \varepsilon| + b \tag{13}$$

The ε is important to avoid when the input data is 0.
As seen previously for linear regression, the optimal values of w and b can be determined by minimising their values from taking derivatives 3 4.

2.4 Neural Networks

The limitations of linear and polynomial regression motivate the use of neural networks, which can approximate complex non-linear functions while maintaining the general trend for unseen data [12]. A neural network is composed of layers of interconnected units called neurons, inspired by the structure of biological neural systems. For a 3-layer network, this is shown as:

$$y = f_{NN}(\mathbf{X}) = f_3(\mathbf{f}_2(\mathbf{f}_1(\mathbf{X}))) \tag{14}$$

Where $\mathbf{f}_1$, $\mathbf{f}_2$ and $\mathbf{X}$ are vectors, and y is a scalar. Here, $\mathbf{f}_1$ and $\mathbf{f}_2$ take the following form:

$$f_n(\mathbf{z}) = g_l(\mathbf{wz} + B_l) \tag{15}$$

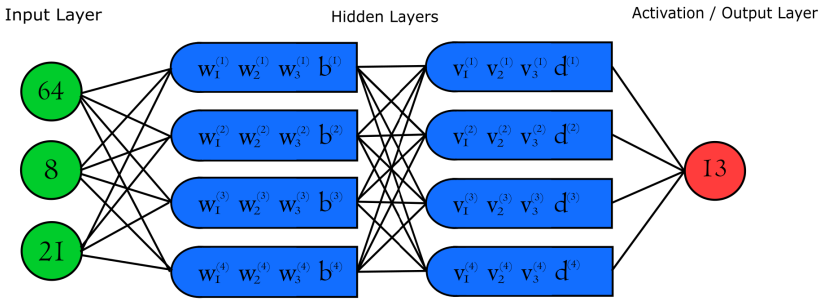


Figure 1: A diagram showing the structure of a basic 4-layer neural network. Each node represents a neuron of the network and each line illustrates a connection between these neurons.

Where l is called the layer index and can span from 1 to any number of layers, g_l is called an activation function (which will be discussed later). The parameters $\mathbf{W}_l$ (a matrix) and $\mathbf{B}_l$ (a vector) for each layer are learned using the familiar gradient descent by optimising, depending on the task, a particular cost function (such as MSE).

This network we employed consists of:

- An input layer - S neurons for N inputs for network
- Hidden layer 1 - Activation
- Hidden layer 2 - Activation
- Output layer – S single neuron for the single output fit of the network

This architecture is called fully-connected. A neural network can contain fully-connected layers - layers whose units receive the outputs of each of the units of the previous layer as input data.

Activation Functions

An activation function is a mathematical function used in a neural network to transform a neuron's input into an output. It is what introduces nonlinearity into the model, allowing the network to learn complex patterns. The choice of activation function significantly affects the learning behaviour and predictive performance of a neural network. One of the goals of this paper is to understand the differences in the application of each function.

ReLU is computationally simple and avoids the vanishing gradient problem for positive inputs. As a result, it has become the standard activation function for hidden layers in many deep learning models.

$$ReLU = \begin{cases} 0 & \text{if } z > 0, \\ z & \text{if } otherwise \end{cases} \tag{16}$$

To use ReLU effectively we must alter it for He initialisation (which then becomes Leaky ReLU): we initialise the weights of each neuron by multiplying them by $\sqrt{\frac{2}{\mathbf{Z}}}$.

The Sigmoid function maps input values into the range 0 to 1, making it historically useful for probabilistic outputs and binary classification tasks. Unlike ReLU, Sigmoid neurons suffer from the vanishing gradient problem, where gradients become very small for large positive or negative inputs; slowing learning in deeper networks.

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{17}$$

Compared with Sigmoid, Tanh is 0-centered, which often improves gradient flow during training.

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \tag{18}$$

Tanh provides an output between 1 and -1, giving it more nuance than the Sigmoid function.

Backpropagation

Backpropagation is the name given to the process of updating the values of parameters in neural networks. It is an efficient algorithm for computing gradients in neural networks using the chain rule. During gradient descent, the neural network's parameters receive an update proportional to the partial derivative of the cost function with respect to the current parameter, in each iteration of training.

In gradient-based optimisation, each parameter is updated in proportion to the partial derivative of the cost function L with respect to that parameter. For a weight $W^{(l)}$ in layer l, the update rule is

$$W^{(l)} \leftarrow W^{(l)} - \alpha \frac{\partial L}{\partial W^{(l)}} \tag{19}$$

and similarly, for a bias $b^{(l)}$:

$$b^{(l)} \leftarrow b^{(l)} - \alpha \frac{\partial L}{\partial b^{(l)}} \tag{20}$$

Here, α is the learning rate, controlling the step size of each update. For a neuron in layer l, the activation is given by

$$a^{(l)} = f(z^{(l)}) = f\left(W^{(l)} a^{(l-1)} + b^{(l)}\right) \tag{21}$$

where $a^{(l-1)}$ is the activation vector from the previous layer, and f is the activation function. Backpropagation computes the gradient of the loss with respect to $z^{(l)}$ and propagates it backward through the network, allowing the efficient calculation of $\frac{\partial L}{\partial W^{(l)}}$ and $\frac{\partial L}{\partial b^{(l)}}$ for all layers.

3 Stock Data

3.1 Stock Specific Tailoring

In order to model the stock market, we imported the YFinance dataset from the appropriate Python library. This is a simple to use, readily accessible dataset, with price data at different time intervals (every second to every week) and a range of stocks to choose from. In order to apply neural networks to stock market data, there were some steps and choices that had to be taken to model data for the most predictive power.

3.2 Returns

Financial markets generate large volumes of time-series data in the form of stock prices, typically represented as Open, High, Low, and Close (OHLC) values for each trading day. To minimise the inherent volatility of stock market data, it is best to train data based on returns and in turn predict returns, defined as the relative change in stock price over a future period, rather than cumulative price.

$$R_{t+f} = \frac{P_{t+f} - P_t}{P_t} \tag{22}$$

Where R_{t+F} is the return, P_t is the closing price at time and f indicates the time in the future we want to predict.

Returns provide a scale-independent representation of price movements, allowing meaningful comparisons between assets with different absolute price levels. For example, a £1 increase has very different significance for a £10 stock compared to a £1000 stock, so returns normalise this change. This makes returns a more appropriate variable for statistical analysis and machine learning models.

Using returns instead of raw prices helps reduce scale effects, improves the statistical properties of the input data, and allows the model to focus on predicting relative price movements, which are typically more relevant for trading and investment strategies.

3.3 Look-back Window

A look-back window is a sequence of past observations used as input features to predict future outcomes. In this project, a 10-day look-back window is used, meaning the model observes the OHLC values of the past 10 days to predict the return on the next day. Formally, if $\mathbf{x}_t$ represents the vector of OHLC values at time t, the input feature vector for each training example can be written as:

$$\mathbf{x}_t = (X_{t-5}, X_{t-4}, X_{t-3}, X_{t-2}, X_{t-1}) \tag{23}$$

Using a look-back window enables the neural network to learn relationships between recent price behavior and subsequent market movements. The choice of window length represents a trade-off: shorter windows capture only very recent information, while longer windows may incorporate more historical context but increase model complexity and the dimensionality of the input space. A 10-day window provides a balance between capturing short-term market dynamics and maintaining a manageable feature size for training.

4 Image Processing

A key advantage of a neural network over other forms of machine learning, is the capability of the network to intake large inputs of data and analyse all elements of that input simultaneously. Such a task, that clearly demonstrates this superiority, is classifying the contents of an image. Digital images are composed of pixels, the full information of each pixel (brightness, colour, position within the image, etc) can be stored as simple numbers on a computer and then be perfectly recreated. The contents of a picture only become recognisable to a human when many pixels are assembled together, the same is true for machine learning algorithms. This necessitates the storage of large amounts of data, which is an ideal environment for a neural network to show its effectiveness.

4.1 The MNIST and Fashion MNIST datasets

The MNIST dataset [13] and the similar Fashion MNIST [14] dataset, are lists of 2D arrays of numbers which describe handwritten numerical digits and types of clothing respectively. Each set contains 70,000 entries, 60,000 'training images' the neural network uses to learn the classes and a further 10,000 'testing images' used at the end of the process to assess how well the model can classify each image. Every image is composed of 784 pixels (A resolution of 28×28 for each image) and is already matched to a class, contained in a separate list. For MNIST, each class is a number paired with its digit (i.e. class [0] for digits of 0, class [1] is for digits of 1, etc.) and likewise Fashion MNIST does the same but for 10 different types of clothing. In the following sections, we will teach both a linear regression model and a neural network with the training data provided by either dataset. Then an attempt to measure the performance can be made, using two different metrics: the amount of time to complete the task and the accuracy with which the correct testing images were selected.

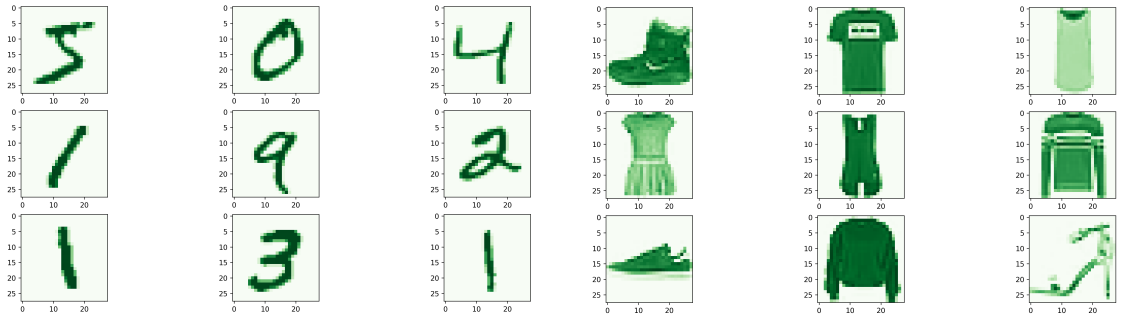


Figure 2: Two sets of nine images from the MNIST dataset (handwritten digits) and the Fashion MNIST dataset (pictures of clothing). All images are 28×28 pixels, with every pixel only containing a value for brightness. Fashion MNIST is generally considered harder as similar looking items can belong to separate groups (such as the ankle boot, sneaker, and sandal).

It is important to avoid using the outputted loss of either model to directly compare to one another, as the models utilise different loss equations (MSE for linear regression and CCE for the neural network). Doing so would be meaningless, but we can comment on the value of the loss function in comparison to the performance of that model on other tasks.

4.2 Image Pre-processing

Before a machine learning algorithm can attempt to recognise patterns inside a picture, it first needs to be able to 'see' the image. This 'seeing' problem occurs due to the inherent two-dimensional nature of images, which is incompatible with the one-dimensional input needed for the neural network. This problem could compound with additional informational nuances about colour or position. Thankfully, the MNIST datasets [13][14] we discussed previously can be manipulated in such a way that the need for more advanced methods of image pre-processing (such as a convolutional neural network) are unnecessary to establish neural network superiority. Each pixel has an assigned brightness value and, when flattened into one-dimensional vector, is of a consistent order (meaning a chosen element of one vector from an arbitrary image, is in the position of the same element from a different image).

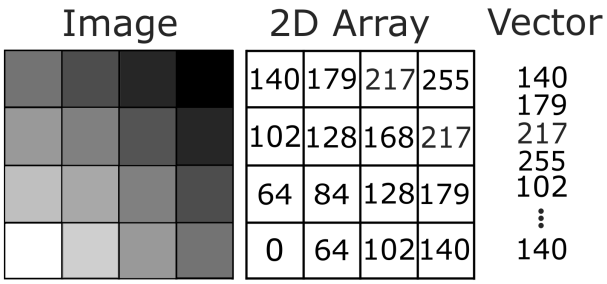


Figure 3: A part of the pre-processing for a machine learning algorithm. A value between 0 and 255 is assigned to every individual pixel and is then 'flattened' into one-dimensional vector.

Now that the image can be inputted to the neural network, a final normalisation step is included; directly dividing all values in the vector by 255 which forces all individual values inputted to range between 0 and 1. Normalisation can prevent weights or biases inside the algorithm from 'blowing up' and potentially destroying all useful outputs of the network.

4.3 The Softmax Activation Function

The Softmax activation function is primarily used in multi-class classification such as the MNIST dataset. It is the generalised form of the previous Sigmoid function, now accounting for multidimensional data[12]. Unlike its counterpart, the Softmax function is typically only applied to the final output layer of a network. Since it is an activation function that can take a vector input function and output a vector where all its values range from 0.0 to 1.00 and $\sum y_i = 1$.

$$y_i = \frac{e^{x_i}}{\sum_{i=1}^{c} e^{x_i}} \tag{24}$$

Where c is the amount of output neurons. Given this property, the function is used as a method to ascertain a network's confidence. Care must be taken when handling an exponential function, such that the inputted values don not cause an error by becoming to large in value. This is mitigated greatly with an initial normalisation step ensuring that for most cases values will remain small.

5 Stock Data Results

In order to analyse the efficacy of neural networks for predicting patterns in financial markets, we have built, trained and experimented a standard neural network consisting of an input layer, two hidden layers, and an output layer.

In order to have the highest accuracy in the network, the input layer was chosen to have a number of neurons equivalent to the number of input datasets (this being four for financial data, for the open, high, low, and close data points), with 64 and 16 neurons in the two hidden layers, respectively, and a single neuron in the final layer (to create a single fit to the data). Furthermore, a learning rate of $\alpha = 0.0005$, a batch size = 64, a forward shift = 1 day, and a look-back window = 10 was used over 2000 epochs, usually studying Apple's financial data - with Stochastic Gradient Descent as the optimiser function. That is, the networks were trained iteratively with 64 data points to fit to the same financial data one day in the future.

Additionally, we trained each algorithm consistently on 80% of the input data, hence testing on 20% of it; with lower training values reducing the accuracy of the neural network's fit substantially, whilst higher training percentages gave rise to errors as the network overfit its predictions.

Alongside these neural network implementations, a maximum-likelihood linear regression model was constructed as a baseline for numerical prediction. The purpose of including this model is twofold. First, it provides a classical reference point against which the gains of more complex architectures can be judged. Secondly, it allows the stock market task to be discussed in terms of interpretability as well as predictive strength.

To compare the efficacy of each activation function within the hidden layers of the neural network, multiple tests were ran:

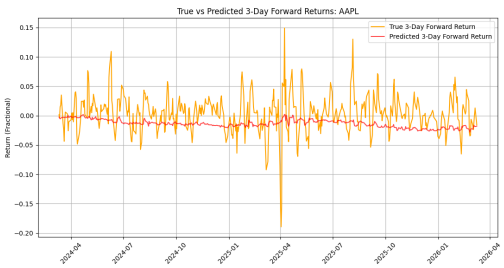
- **Test A** uses ReLU activation twice: once in each of the two hidden layers.
- **Test B** uses Sigmoid activation twice: once in each of the two hidden layers.
- **Test C** uses Tanh activation twice: once in each of the two hidden layers.

5.1 Neural Networks

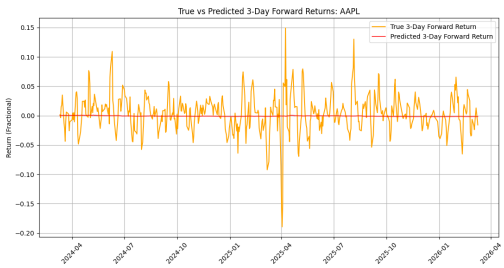
The first and most obvious measure of the neural network's capacity to understand and predict financial returns is to directly compare its output to the actual 3-day forward returns.

Returns

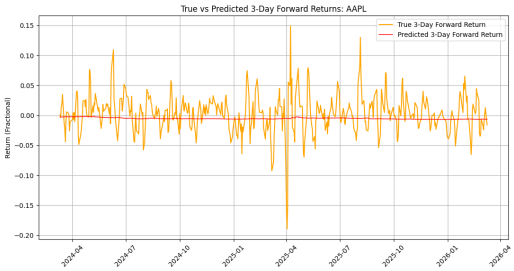
As discussed in section 2.5.2, a return is defined as the difference between the opening and closing price of a particular stock. Here, the past two years of returns for Apple has been predicted and compared to the real values.



(a) Test A: ReLU Activation



(b) Test B: Sigmoid Activation



(c) Test C: Tanh Activation

Figure 4: Predicted returns compared to their real counterparts graphs, for each activation function, over the past two years.

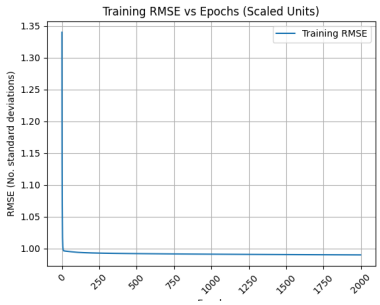
The first and most obvious difference that can be made is that ReLU incorporates much more variation in between each return window. This is largely due to its ability to mitigate the vanishing gradient problem, allowing gradients to propagate effectively through deeper layers [17]. As a result, the network trained more efficiently and captured short-term trends in the financial data more effectively. However, some instability was observed, likely due to the noisy and non-stationary nature of market data, which can cause large gradient updates.

By comparison the Sigmoid function predicts almost no returns constantly, throughout the two-year testing period. While the Tanh function adds slightly more variation, it still shows more commonality with the the results from Test B. This can be attributed to its tendency to saturate for large positive or negative inputs, leading to very small gradients during backpropagation [17]. Unlike Sigmoid, Tanh is zero-centered 18, which improves gradient flow and helps with optimisation. This allowed the network to model both upward and downward price movements more effectively.

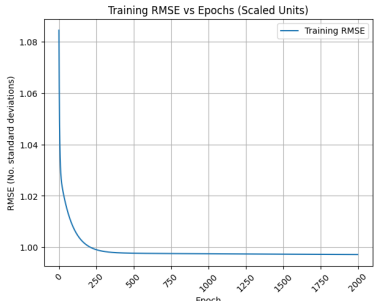
Perhaps the first observation one could make from comparing these graphs, is that none of them capture the volatility of the market with any rigour. While returns naturally oscillate around zero, the predictions lack the dramatic, irregular and rapid peaks or troughs seen week to week in financial markets. This suggests that the networks struggle to learn high-frequency fluctuations, likely due to limitations in model complexity, the choice of input features, or the inherent noise and stochastic nature of financial markets. In spite of this, each function was able to capture the trend of the particular stock, to varying strengths: ReLU clearly the best, and, much further behind, Tanh then Sigmoid.

Root Mean Square Error

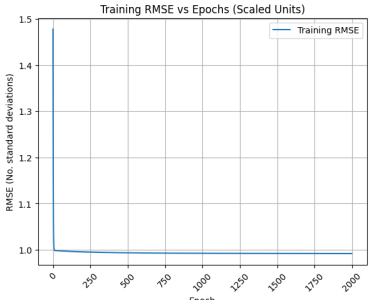
Another important measure of accuracy is to understand how the root mean squared error ($\sqrt{l}$), described in equation 2 changes over the number of epochs. The RMSE is the standard deviation (σ) of the predicted values, providing an interpretable measure in return units.



(a) Test A: ReLU activation



(b) Test B Sigmoid Activation



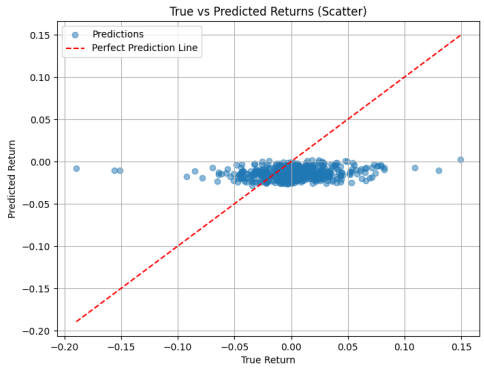
(c) Test C: Tanh Activation

Figure 5: RMSE (standard deviation) decreases exponentially across epochs

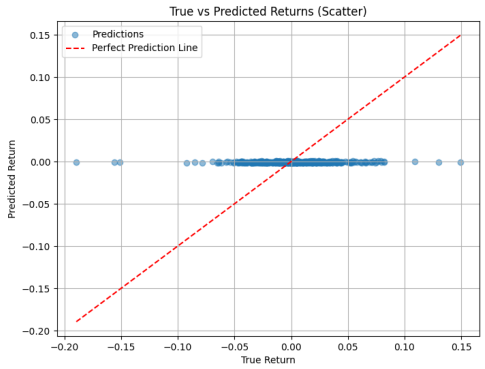
Each graph in Figure 5 paints almost the exact same picture with regard to the root means squared error: a high start with an exponential decrease to a value just under one. Training until 2000 epochs almost entirely ensures that σ flatlines. The error in the graph for Test B begins at a lower value - most likely arbitrarily due to the random selection weights during initialisation. Another difference with the Sigmoid function is that it is the slowest, while still incredibly quick, to plateau.

General accuracy

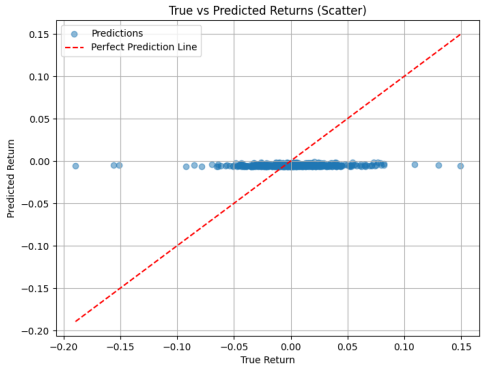
A more visually explanatory picture is painted by the following scatter graphs in Figure 6, contrasting the values of the predicted returns against the true values. The dotted red line across the diagonal marks the points at which the predicted return exactly matches the real returns. Hence, points further away from this line signify less accurate predictions. A horizontal line of points indicates the model is predicting a constant return regardless of input. A vertical line of points indicates the model is predicting a wide range of returns regardless of input. A perpendicular line of points indicates the model is predicting returns that are uncorrelated with the true returns.
The graphs have much in common, most distinctly, is that the points are normally distributed horizontally, crossing the diagonal at slightly negative values in both the x and y axes. This demonstrates again that each activation function allows the model to capture the general trend of the returns, albeit to differing degrees.



(a) Test A: ReLU Activation



(b) Test B: Sigmoid Activation



(c) Test C: Tanh Activation

Figure 6: Scatter plots comparing predicted values to the actual values.

The ReLU function, unmistakably, displays much more clustering than the other two functions. This indicates that, while noisy, the model has learned some relationship between inputs and outputs and is capable of responding to variations in the data. The spread around the diagonal also reflects its greater sensitivity to changes in input, consistent with the higher variability observed in its time-series predictions.

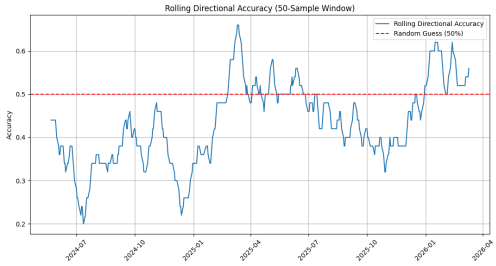
Both of the other functions result in a largely similar level of accuracy to each other, from this analysis. There are much narrower (for the Tanh function, minimally less narrow than Sigmoid) horizontal bands of normally distributed points, just below zero on the x and y axes. Despite Tanh mitigating some of the compression seen in Sigmoid due to its zero-centred nature, both effectively output a near-constant return, largely independent of the input features.

These visualisations [5, 6] highlight the inherent difficulty of financial prediction: even the best-performing model produces a broad scatter, underscoring the low signal-to-noise ratio characteristic of financial markets.

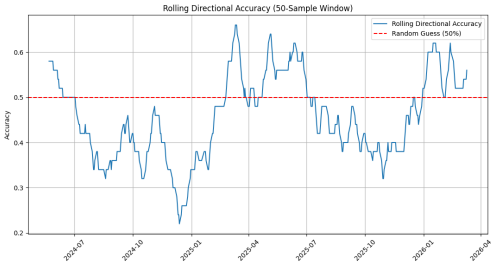
Directional Accuracy

The directional accuracy, which measures the proportion of instances in which the model correctly predicts the sign of the return rather than its exact magnitude, is another helpful tool in understanding the role of different neural networks in predicting financial returns. Here (7), the horizontal, dashed, red line indicates a random guess whether the stock will give a positive or negative return. The blue line shows how the directional accuracy changes through the prediction

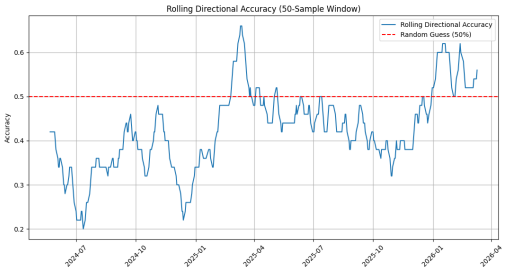
window.



(a) Test A: ReLU Activation, with between 43.8% to 54.4% directional accuracy



(b) Test B: Sigmoid Activation, with between 46.2% to 49.7% directional accuracy



(c) Test C: Tanh Activation, with between 43.0% to 52.2% directional accuracy.

Figure 7: Predicted returns compared to their real counterparts graphs, for each activation function, over the past two years.

Each graph commonly displays a directional accuracy below 50%, which is worse than a random guess. Due to the randomisation of initialization weights, there is some variation in the directional accuracy.

In contrast to what has been seen earlier in this report, the ReLU function has the weakest directional correlation, which could be due to an analytical error, that will be explained later.

Adding to this controversy, the Sigmoid function gave the best directional accuracy, still hovering under a random guess. This is consistent with its near-constant predictions observed earlier: if the model outputs values clustered tightly around zero, small fluctuations in true returns will frequently result in incorrect directional predictions.

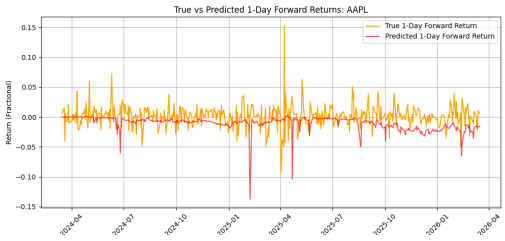
It is also important to note that directional accuracy values across all models remain relatively modest. This reflects the intrinsic difficulty of financial forecasting, where price movements are influenced by a complex combination of stochastic processes, external events, and market sentiment. Even small improvements above random performance can be significant, but the overall results suggest that the models are only capturing a limited portion of the available predictive structure.

5.2 Polynomial & Logarithmic Regression

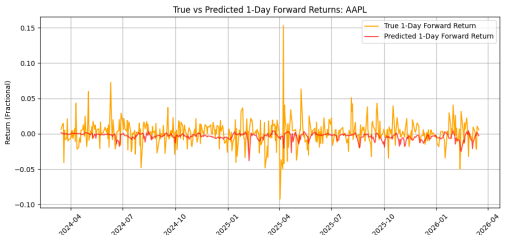
A useful extension for linear regression is both polynomial 12 and logarithmic 13 regression. These more complex regression types allow for better predictions. Polynomial regression has turning points allowing the peaks to be better represented by the predicted results. Logarithmic allows for decrease in gradient and allows smooth curves to be predicted. Here the effects of replacing linear regression in a neural network with polynomial and logarithmic regression is

tested. To see whether the extensions of linear regression increase the accuracy of the predictions of the stock market.
To evaluate the efficacy of polynomial and logarithmic regression, the same analytical methods will be exercised.

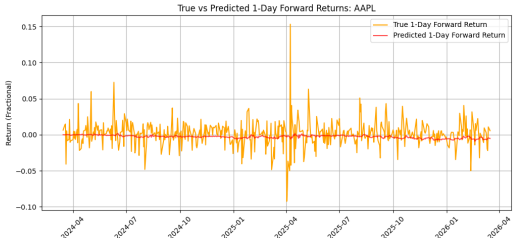
Returns



(a) Quartic Regression



(b) Cubic Regression



(c) Quadratic Regression

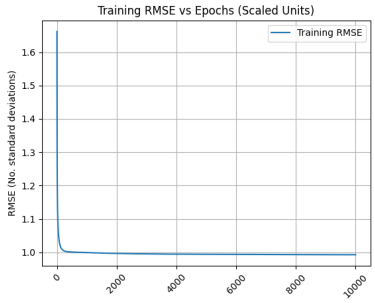
Figure 8: Predicted returns vs true returns over a 3 day period using different order polynomials for polynomial regression.

Figure 8 shows the comparison between 2nd, 3rd, and 4th-order polynomial regressions - revealing a clear progression in model behaviour as the degree of the polynomial increases. As this order rises, the model gains additional flexibility, allowing it to more closely follow fluctuations in the data. In particular, higher-order polynomials attempt to capture the peaks and troughs of the return series with increasing precision, leading to a visibly tighter fit to the training data. However, this increased flexibility comes at a cost. The quartic (4th-order) regression exhibits clear signs of overfitting, as it begins to model not only the underlying trend but also the noise inherent in financial time series. This results in erratic behaviour in the predictions, including sharp and seemingly unmotivated drops in predicted returns.

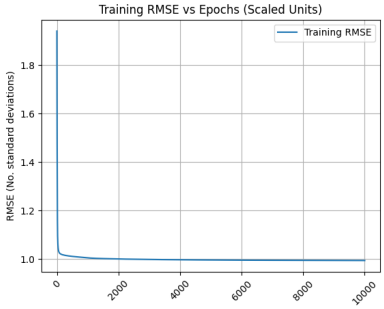
the quadratic (2nd-order) and cubic (3rd-order) regressions display more stable behaviour. The quadratic model, being the simplest, provides a smoother approximation of the data and avoids extreme fluctuations, but at the expense of failing to adequately capture more complex structures such as sharper peaks and troughs. This results in a degree of underfitting, where important features of the data are smoothed out.

Root Means Squared Error

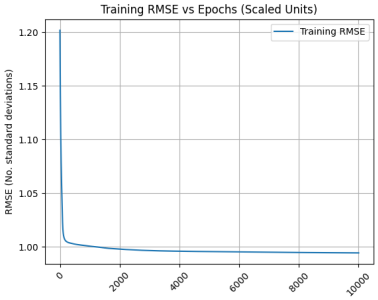
Figure 9 shows the graphs for RMSE, using polynomial regression, giving essentially the same pattern as that of neural networks. One difference is that for quadratic the RMSE appears to go slightly lower than the other methods at higher epochs. Another difference between the regression types is the initial error at the start of the fit. For cubic, this starts higher, with quartic regression giving a smaller error, and quadratic regression minimising this the most. Notably, the degree of this error is arbitrary, and changes based on the initialisation of weights.



(a) Quartic Regression



(b) Cubic Regression



(c) Quadratic Regression

Figure 9: Graph showing how the RMSE value changes with the number of epochs done

General Accuracy

We can analyse the scatter graphs above using the same metrics we used for neural networks.

Figure 10 shows that for the quadratic regression, the scatter plot typically shows a relatively tight clustering of points around a narrow range of predicted values, with only weak alignment along the diagonal. This suggests that the model is underfitting the data, failing to capture the full variability of returns.

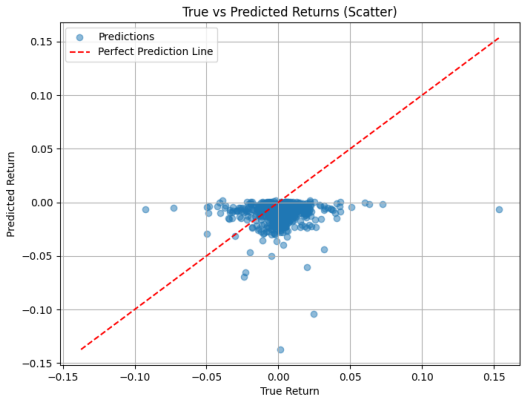
The cubic regression demonstrates a noticeable improvement. Its scatter plot shows a broader spread of points and a clearer, though still imperfect, alignment with the diagonal. This indicates that the model is better able to capture variations in the data and has learned a more meaningful relationship between predicted and actual returns. The increased dispersion reflects a greater sensitivity to input features, allowing the model to respond more effectively to changes in market conditions without collapsing into near-constant predictions.

In contrast, the quartic regression produces a much more irregular and dispersed scatter pattern. While some points may lie closer to the diagonal, the overall distribution is less structured, often appearing noisy or erratic. This reflects the overfitting observed in the time-series results: the model is capturing noise rather than true signal. As a result, predictions can vary wildly even when true returns do not, leading to a weaker overall correlation and reduced reliability

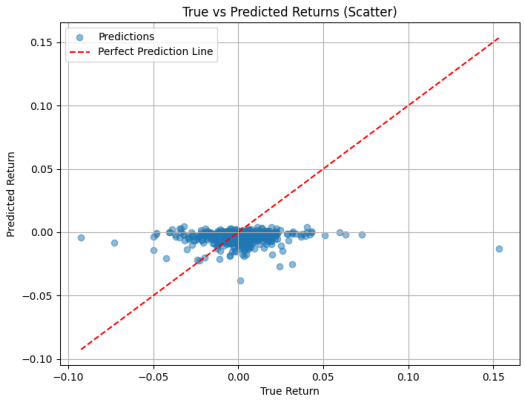
Directional Accuracy

Quartic: 42.3% to 47.8%
Cubic: 46.5% to 53.4%
Quadratic: 44.5% to 52.0%

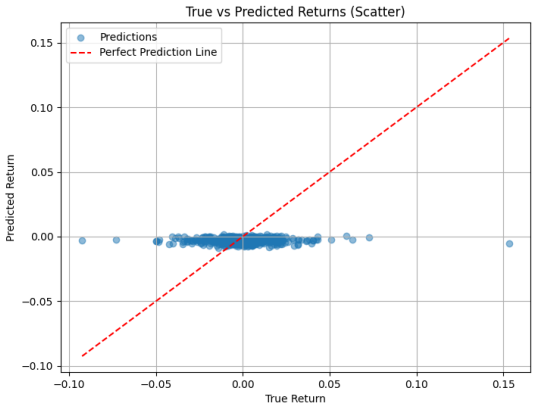
Typically the directional accuracy of cubic was slightly higher than that of quadratic. However, occassionally the directional accuracy of quadratic was higher than that of cubic.



(a) Quartic Regression



(b) Cubic Regression



(c) Quadratic Regression

Figure 10: Scatter plot comparing predicted values with the actual values

This is because the higher-order cubic still had the potential to overfit for some weight initialisations. Quartic maintained the lowest directional accuracy due to its higher odds of overfitting.

5.3 Maximum Likelihood Linear Regression

Returns

As discussed in Section 2.1, the model is fitted by minimising the negative log-likelihood, which is equivalent to minimising the error term ε_t under the assumption that the residuals are normally distributed. As previously explained, the model is used to predict one-day ahead log return rather than raw prices, since returns are more suitable for statistical modelling and are closer to stationarity.

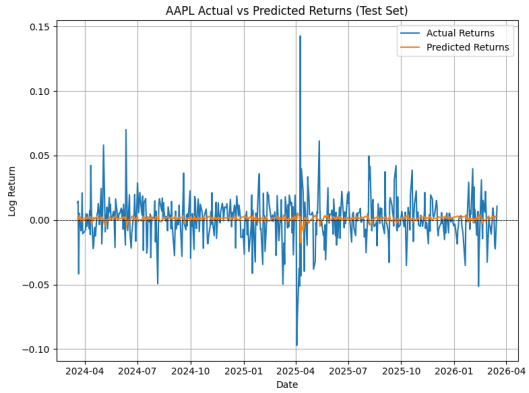


Figure 11: Actual and predicted one-day-ahead log returns for the MLE linear regression model over the test period.

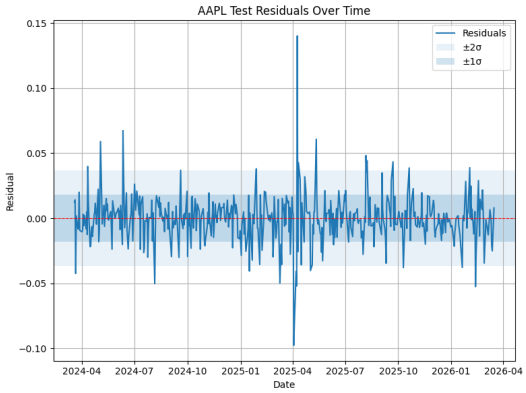


Figure 12: Residuals of the MLE linear regression model over the test period.

Figure 11 shows the actual and predicted returns over the test period. The model is able to follow the general movement of returns in some places, but it does not capture the more extreme fluctuations particularly well. This is expected, since daily stock returns are highly noisy and much of their variation is difficult to explain using a linear model with a Gaussian error structure. Figure 12 shows the residuals (the difference between the testing data and the prediction) across the test period. Ideally, these residuals should behave like random noise centred around zero. While they do fluctuate around zero overall, periods of larger deviation remain visible, suggesting that the model does not fully explain all the structure present in the data.

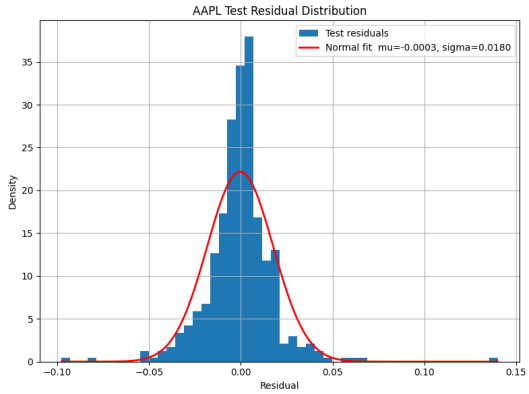


Figure 13: Distribution of the model residuals, shown with a Normal fit for comparison.

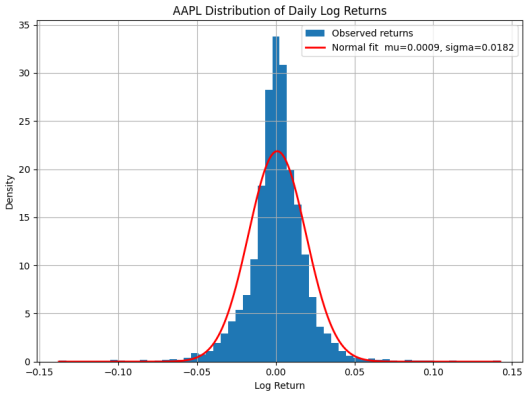


Figure 14: Distribution of the observed log returns, shown with a Normal fit for comparison.

Figure 13 and Figure 14 show the distributions of the residuals and log returns, respectively. In both cases, comparison with a Normal fit helps assess whether the Gaussian assumption used in the likelihood function is reasonable. The return distribution exhibits heavier tails than a perfect Normal distribution, which is common in financial data, while the residual distribution is somewhat more concentrated but still departs from normality in the tails.

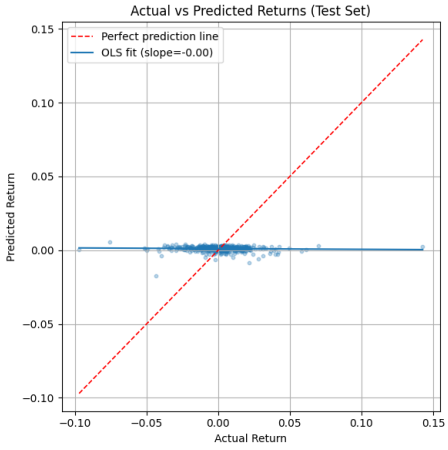


Figure 15: Scatter plot of predicted returns against actual returns, with fitted line, showing the relationship between model predictions and observed values.

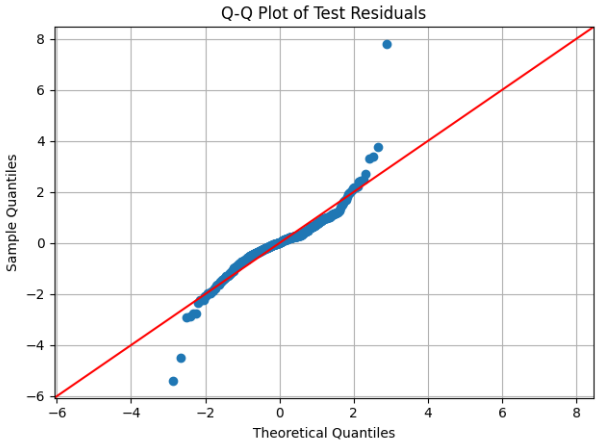


Figure 16: Q–Q plot of the residuals for the MLE linear regression model.

Figure 15 provides a scatter plot of predicted returns against actual returns, together with a fitted line. If the model were highly accurate, the points would cluster closely around the line $y = x$. Instead, the points remain quite dispersed, indicating that although there is some relationship between prediction and outcome, the predictive power remains limited. This is consistent with the residual behaviour already seen in Figure 13, where departures from normality remain visible in the tails. The same feature is reflected in Figure 16, which shows that the residuals deviate from the reference line at the extremes. This suggests that extreme movements are not captured especially well by the model.

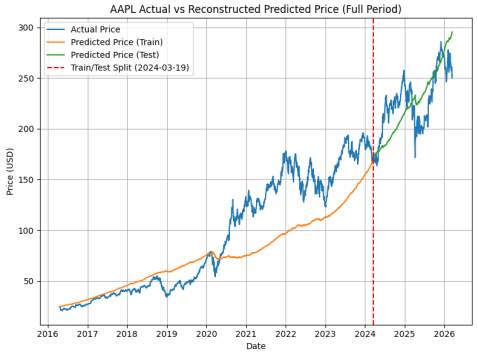


Figure 17: Reconstructed predicted price path compared with the actual price series.

Finally, Figure 17 reconstructs the predicted price path from the predicted returns and compares it to the true price series. This gives a more intuitive picture of how prediction errors accumulate over time. Although the predicted path broadly follows the direction of the true price movement, discrepancies increase over longer periods, reflecting the difficulty of using one-step return predictions to recover the full price trajectory accurately.

Root Mean Squared Error

The RMSE was used to measure the average size of the prediction errors, with larger errors penalised more heavily because of the squaring in the Mean Squared Error. This makes RMSE useful when large deviations are especially undesirable, but it also means that the metric can be strongly affected by a relatively small number of extreme market movements. In financial return

series, where abrupt changes and heavy tails are common, the RMSE may therefore appear relatively large even when the model captures the general behaviour of the series reasonably well on more typical trading days.

Model	RMSE	Directional Accuracy
Mean Baseline	0.017834	55.3106%

Table 1: Baseline model performance measured using RMSE and directional accuracy.

General Accuracy

The general accuracy measure also depends on RMSE, so it inherits the same bias towards larger errors. As a result, a model may appear less accurate overall because it fails to predict a small number of extreme movements, even if it performs reasonably well across most observations (see fig. 17). In addition, since average daily returns are often close to zero (fig. 14), a simple model based only on the mean return may sometimes appear deceptively competitive under error-based measures. For this reason, general accuracy should not be interpreted in isolation, but considered alongside directional accuracy and residual analysis.

Directional Accuracy

Directional accuracy measures the proportion of occasions on which the model correctly predicts the sign of the return, that is, whether the market moves up or down. This is particularly relevant in financial applications, where correctly predicting direction may be more valuable in practice than predicting the exact size of the return.

A model may therefore have a modest RMSE while still achieving useful directional performance, or instead produce numerically close predictions that fail to identify market direction consistently. Directional accuracy thus provides a complementary perspective on model performance alongside magnitude-based error measures.

6 Temperature Data Results

As well as testing the efficacy of neural networks on financial data, we also used data on rises in global temperature to observe how the networks performed in smaller datasets with greater relative fluctuations. To do this, we used the NOAA Global Surface Temperature dataset (NOAAGlobalTemp)[20] and the Berkeley Earth Surface Temperature dataset (BEST)[21] as inputs for the network, and the Met Office's Hadley Centre/Climatic Research Unit Global Surface Temperature data set (HadCRUT5)[22] as target data. These datasets each contain predictions for changes in global temperature from 1850 to 2025 in degrees centigrade ($^{\circ}C$); resulting in 176 total data points for the network to train and test on.

Due to high imprecision when determining changes in global surface temperature, these datasets possess high fluctuations between points which, when paired with the substantially small size of the datasets, makes this data more inherently challenging to train a neural network against. This is rectified by using deeper hidden layers in the network.

Using the same logic as in Section 5, to ensure the highest accuracy in the network we set the input layer to have a number of neurons equivalent to the number of input datasets (which is 3 in this case). As these datasets only contain yearly data, deeper hidden layers are required in the network to maintain accuracy; hence we have 64 neurons in the first hidden layer, and 32 in the second layer. To ensure one fit to the data, the final output layer has a single neuron. Furthermore, the hyperparameters of the network are initialised just as in Section 5 - with a learning rate of $\alpha = 0.0005$, a batch size $= 64$, and 2000 epochs being used.

6.1 Neural Networks

As seen previously in section 5.1, varying the activation functions used in the network's hidden layers greatly impacts results. Hence, we ran the network through the following tests:

- **Test A** uses ReLU activation twice, for each activation layer.
- **Test B** uses Sigmoid activation twice, for each activation layer.
- **Test C** uses Tanh activation twice, for each activation layer.

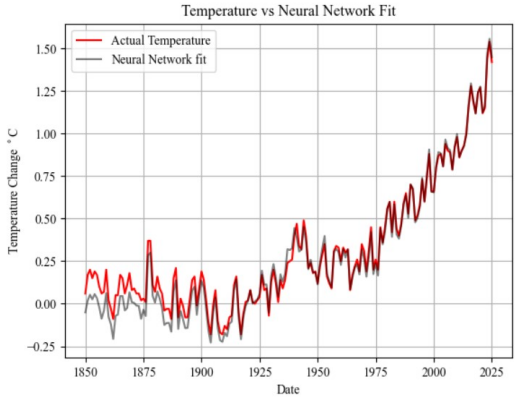


Figure 18: Test A: with a neural network with two ReLU hidden layers of 64 and 32 neurons.

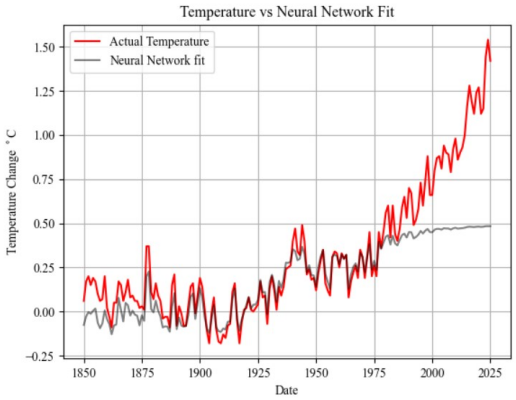


Figure 19: Test B: with a neural network with two Sigmoid hidden layers of 64 and 32 neurons respectively.

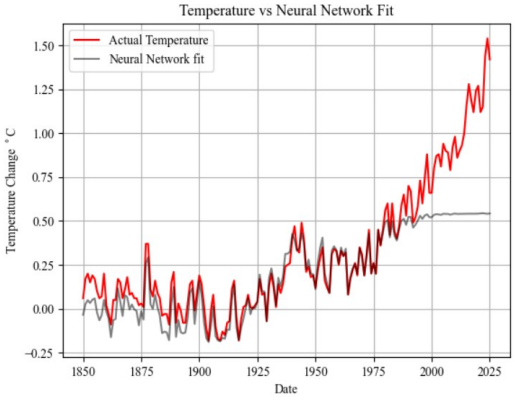


Figure 20: Test C: with a neural network with two Tanh hidden layers of 64 and 32 neurons respectively.

As can be seen in Figure 18, the use of ReLU activation functions in hidden layers greatly improves the accuracy of the fit to the data when compared to the use of Sigmoid or Tanh activation functions (shown in figures 19 and 20 respectively); giving a percentage error = 2.13% for ReLU layers, compared to a percentage error = 50.58% for Tanh layers and a percentage error = 56.64% for Sigmoid layers. This is due to how the ReLU activation function prevents gradients in the network from becoming oversaturated in deeper layers (as is the case here) by maintaining strong gradients throughout. Additionally, by definition, ReLU sets negative gradients to zero; making unnecessary neurons inactive. This not only improves efficiency for larger networks (as only more useful neurons are maintained), but also prevents deviations in the fit due to vanishing gradients (which can be observed in the beginning and ends of the fits in figures 19 and 20).

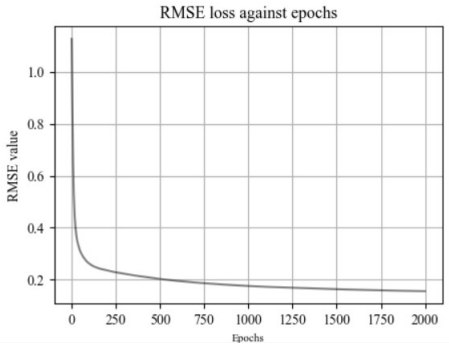


Figure 21: Test A: RMSE loss progression for a neural network with two ReLU hidden layers of 64 and 32 neurons respectively.

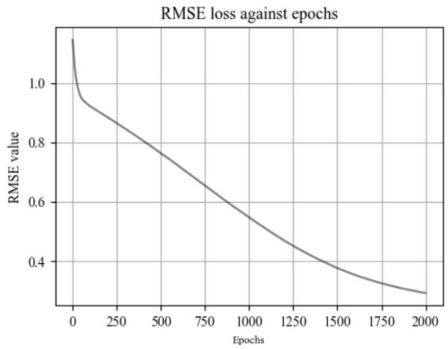


Figure 22: Test B: RMSE loss progression for a neural network with two Sigmoid hidden layers of 64 and 32 neurons respectively.

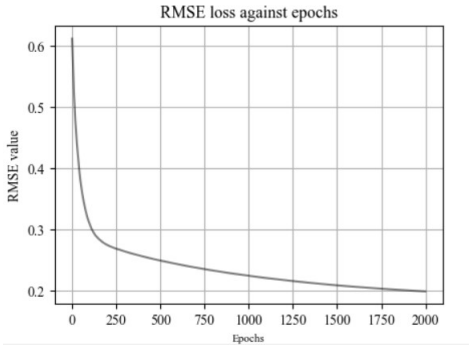


Figure 23: Test C: RMSE loss progression for a neural network with two Tanh hidden layers of 64 and 32 neurons respectively.

For hidden layers, the Tanh activation function is usually superior to the Sigmoid activation function as it is zero-centred; producing gradients between ± 1, instead of between 0 and 1. This allows the function to better mitigate optimisation issues by allowing positive and negative updates - resulting in more balanced updates. The Tanh activation function's steeper gradient relative to Sigmoid also provides it with a faster convergence (as can be seen above in figures 22 and 23 showing how the two functions compare with RMSE loss). As explained earlier, ReLU prevents gradients from becoming oversaturated in deeper networks such as this one, maintaining a strong RMSE loss over iterations - which is accurately observed in Figure 21.

6.2 Linear, Polynomial, and Logarithmic Regression

As observed earlier in Section 5.2, investigating the effects of how different regression algorithms perform against the neural network can yield interesting results. Here, we test three regression algorithms on the temperature data; linear, 2nd-order polynomial, and pure logarithmic regression. These provide a unique view into how these regression types behave on smaller datasets, the effects of overfitting, and provide a retrospective view of the limitations of the neural networks used previously on this dataset.

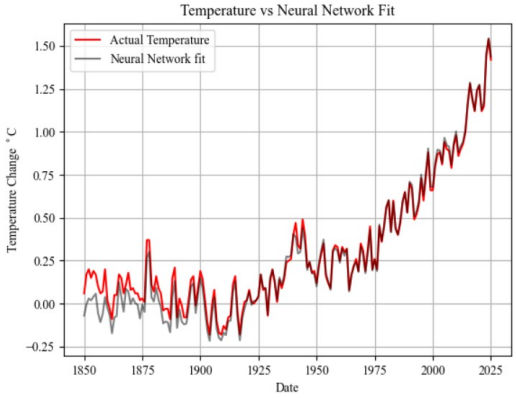


Figure 24: Linear regression is the most accurate, and actually outperforms all neural networks.

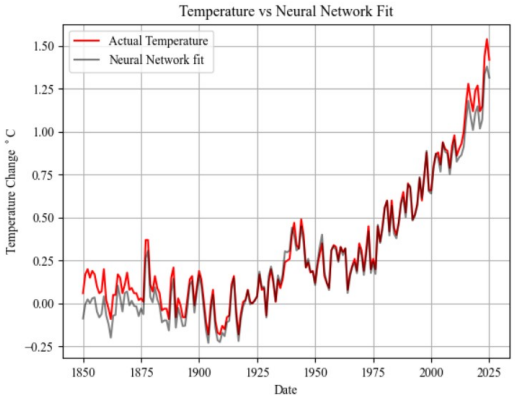


Figure 25: 2nd-order polynomial regression gives a middling accuracy.

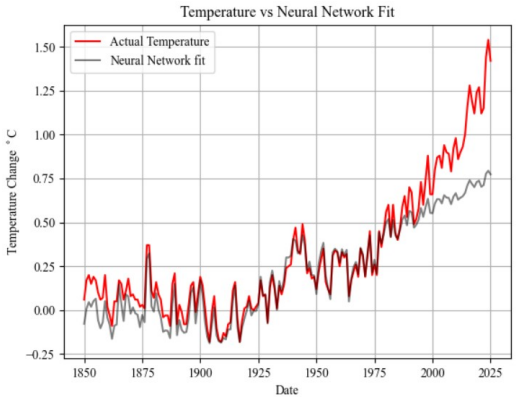


Figure 26: Logarithmic regression gives the lowest accuracy of the three.

As can be seen in Figure 24, linear regression performs remarkably for this dataset, returning a percentage accuracy of 2.03% (0.1% more accurate than the ReLU neural network used earlier). While at first this may seem surprising, it follows perfectly from the mathematical basis of linear regression 3.3 - its simple definition promotes a better generalisation for the linear regressed fit, preventing overfitting in smaller datasets (like the temperature rise dataset used here). This prevents the fit from overfitting to noise in the data, producing a corresponding greater accuracy.

Polynomial regression, taking a more complex form, is substantially more prone to overfitting; resulting in a less accurate fit with a percentage accuracy of 7.11%. This can be seen in Figure 25 which shows deviations in the start and end of the fit (akin to similar overfitting deviations seen in the Tanh and Sigmoid neural networks earlier). Similarly, logarithmic regression is prone to overfitting, and is most successful for large datasets with positive trends (as negative gradients are omitted from the logarithm of the regression's formula). For a dataset such as this, with regular negative fluctuations due to noise, it fails to produce strong results - giving a percentage accuracy of 37.03% - and deviates greatly for the start and end of the fit (as can be clearly seen in Figure 26).

7 MNIST Datasets Results

In this section we will discuss the accuracy of two machine learning algorithms (linear regression and a 3-layer ReLU neural network) at classifying 10,000 28×28 resolution images provided by the MNIST and Fashion MNIST datasets. As previously commented, the loss of either method will be calculated but will not be compared to one another. Instead, the accuracy (percentage of guesses the algorithm correctly chooses in total) will be compared.

7.1 Linear Regression

Whilst attempting the MNIST datasets with a linear regression model, it is expected that the accuracy of the model will be low. We expect this because the model used directly plots the data of the brightness of each pixel against the pixel indices, formulates a linear function of best fit and then outputs a value. That value is then clipped and rounded to the nearest integer from 0 to 9. This totally ignores the total structure of an image, meaning that the model is blind to emergent patterns from surrounding pixels or even if there are no pixels nearby (such as at the edge of an image).

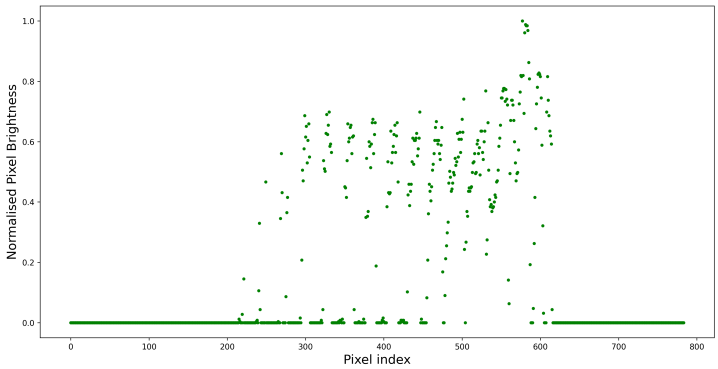


Figure 27: A visualisation of what the linear regression model 'sees' when it classifies an image. This graph plots the normalised pixel brightness, necessary during pre-processing of the image, against the pixel index, which states the position in the input vector the brightness obtained from. The current image displays the ankle boot displayed in the top left of Figure 2

.

Applying this linear regression method to the MNIST dataset resulted in an accuracy score of 23.0% and loss value of 3.151. This is poor result, the model failed to appropriately suit the data. This is evidenced by the large loss value, typically for a well fitting model this would be much closer to 0. Now for the Fashion MNIST dataset, the model attained an accuracy score of 38.6% and a loss score of 1.968. This is a large improvement over the previous dataset as it seems the model was greater able to match the data, achieving a significantly lower loss score. Both results are surprising, given that one would expect the model to perform similar to a model that guessed randomly. Such a model would attain an accuracy score of 10%, but both datasets performed significantly better.

7.2 Neural Network

As previously alluded to, it is expected a neural network's performance will be superior to that of other machine learning algorithms. The image processing neural network used includes a Softmax activation output layer, thus enabling the algorithm to classify images with a more sophisticated method.

Applying the algorithm to the MNIST dataset after 1 Epoch (i.e. only performing a single back pass) achieves an accuracy score of 85.4% and a loss value of 0.572. Doing the same for the Fashion MNIST dataset, it achieves an accuracy score of 74.7% and a loss value of 0.746. Clearly we can see that the neural network performs vastly superior to the linear regression algorithm, however this is still considered a relatively poor result. So to improve the results of the algorithm we simply increase the number of epochs.

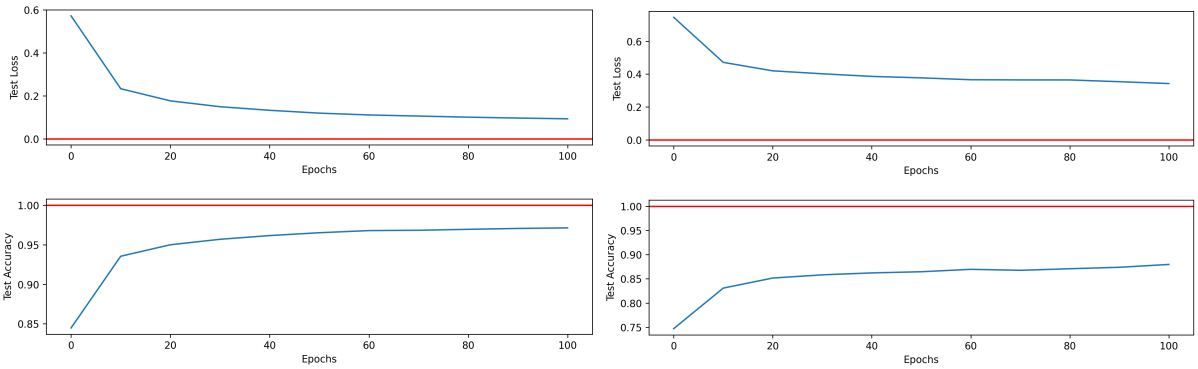


Figure 28: Graphs displaying the improvement of the neural network at classifying images from the training set over 100 epochs. On the left is MNIST and the right Fashion MNIST. The red horizontal line on each graph shows the optimal outcome for each variable (i.e. achieving 100% accuracy or a loss value of 0) which the algorithm is attempting to approach over every epoch.

At 100 epochs, the network achieves accuracy scores of 93.5% and 83.1% with accompanying loss values of 0.233 and 0.472 for the MNIST and Fashion MNIST datasets, respectively. Pushing for a longer training time of 1000 epochs, we can observe from Figure 29 that improvement plateaus. At 1000 epochs the network achieves accuracy scores of 97.1% and 88.0% with loss values of 0.093 and 0.343.

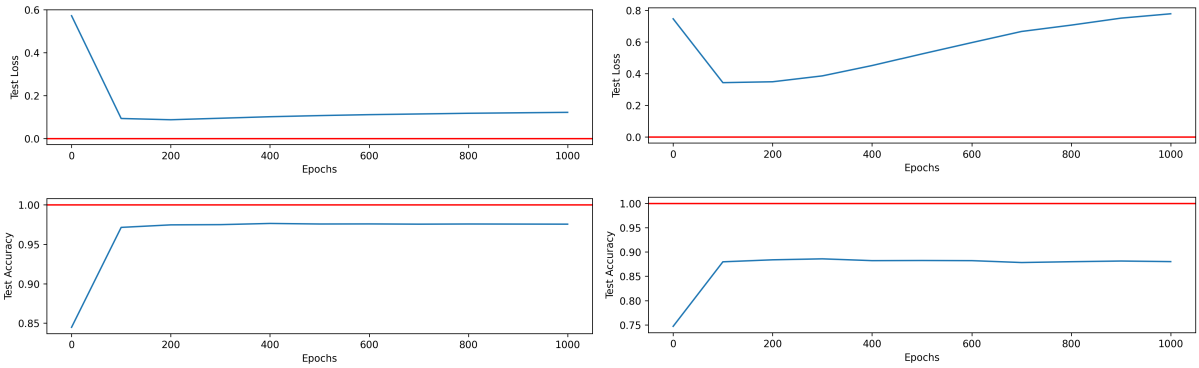


Figure 29: Graphs displaying the improvement of the neural network at classifying images from the training set over 1000 Epochs.

Seeking even further improvement by expanding to 1000 epochs yields only small to no improvement. The algorithm obtains accuracy scores of 97.5% and 88.0% loss values of 0.112 and 0.778. It should be clear that no further improvements to the scores can be made without attempting to use an alternative method.

8 Comparison

After comparing the integration of various activation functions in a neural network, ReLU seems to have an advantage when analysing returns and general accuracy, due to its ability to mitigate

the vanishing gradient problem. However, this conclusion had doubt cast due to the directional accuracy for ReLU being the most volatile and unpredictable.

Linear regression is computationally inexpensive, straightforward, and easily interpretable. At the same time, the limitations of linear regression are equally instructive. The model assumes that the target can be explained as a linear combination of the selected features, and so it cannot naturally capture complex nonlinear interactions. One should therefore not expect the regression model to reproduce the full structure of the data, particularly when compared against a flexible neural network trained on richer feature representations. The linear model performs comparably to the neural network (generalising for activation functions) on some metrics, like directional accuracy, which is statistically meaningful due to its lack of non-linearity.

Polynomial and logarithmic regression is a useful extension for linear regression. However, when used in the neural network it appears to slightly lower the directional accuracy from the base neural network this is probably due to overfitting even in the quadratic regression. Quartic has a massive tendency to overfit causing it to make wildly incorrect productions predicting peaks and troughs when there are none causing it to have a significantly lower accuracy. Cubic is generally the best however it sometimes has a tendency to overfit causing it to occasionally be worse than quadratic.

Comparing different regression types and varying neural networks on global temperature rise data provides highly insightful results. By using the NOAAGlobalTemp [20], BEST [21], and HadCRUT5 [22] temperature datasets, which each contain only 176 data points, the accuracy of these techniques over smaller datasets can be observed. Linear regression performs the best for these datasets, giving a percentage error of 2.03% due to a lack of overfitting to the high noise inherent in the dataset. Using ReLU hidden layers in a deep neural network achieves a near-comparable accuracy of 2.13%, due to ReLU's ability to make unused neurons inactive, followed by accuracies of 7.11% and 37.03% for polynomial and logarithmic regression respectively (which overfit to noise in the data compared to linear regression). Finally, Tanh and Sigmoid hidden layers give accuracies of 50.58% and 56.64% respectively; due to how both these activation functions oversaturate the gradients significantly.

As for the image processing task, the superiority of a neural network over linear regression is evident. A neural network achieved an accuracy score over 70% greater than the linear model (an accuracy score of 97.1%) for the MNIST dataset when trained for 1000 epochs. Contrastingly, this superiority was diminished for the Fashion MNIST dataset; achieving a score of 88.0% after 1000 epochs which was only a 50% improvement. This lessened increase was compounded with the surprising results of the linear model at the task.

The outputs of the linear model for both datasets attained more accuracy than expected. This could be explained by the clipping applied to the final step of model. Before this step the model was allowed to predict any value, including negative values. This could mean the model had a bias to guess values of class [0] (for all potentially guessed negative values) and for class [9] (all potentially guessed values greater than 9). Since the testing images set is composed of 10% images from every class, we should have expected an accuracy closer to 20%. As the model would correctly guess half from each class of image due to the bias. However, the results from the Fashion MNIST dataset suggests this is an incomplete explanation, the model performed significantly better even than this refined expected value.

9 Limitations

The linear regression model developed for the stock market component is subject to several limitations. Most fundamentally, it imposes a linear relationship between engineered predictors

and future returns. This is a strong simplification in a financial setting, where market behaviour is driven by a combination of nonlinear dynamics, structural breaks, and exogenous events that cannot be inferred from recent price history alone. Even when the chosen features are informative, the model is constrained to combine them in a fixed linear manner.

A second limitation lies in the distributional assumptions used for maximum-likelihood estimation. The Gaussian error model provides a clean and analytically convenient framework, but financial returns are well known to deviate through heavy tails and time-varying volatility [19]. Consequently, a satisfactory fit in the mean doesn't ensure that the model captures the data's true structure; hence the necessity of diagnostic testing the residuals.

There are also practical modelling limitations. The fitted model depends heavily on the quality of the engineered input variables, and so its success is partly determined by feature design rather than learned representation alone. The prediction of returns rather than prices introduces a further interpretive subtlety: small one-step return errors can accumulate into large deviations once reconstructed price paths are plotted over long horizons. A visually poor price-tracking figure therefore does not necessarily imply a complete failure of the one-step return model, though it does underline the difficulty of the forecasting task.

From a broader perspective, the principal benefits of linear regression in this project are its interpretability, speed, modest computational cost, and statistical transparency. The principal drawbacks are its inability to capture strong nonlinear structure, its sensitivity to misspecified assumptions, and the limited extent to which simple linear combinations of historical features can explain an inherently noisy financial process.

The polynomial regression developed for the stock section has two major limitations: run time and overfitting. This is because when you increase the order of the polynomial the run time increases exponentially. Overfitting happens more often in higher orders as the network fits too closely to the training data and struggles with predictions for the future.

10 Conclusion

Encapsulating the report's findings, we found that using ReLU activation functions in the hidden layers of the neural network gave the highest accuracy for the YFinance and Temperature datasets - providing a consistently faster convergence in RMSE loss when compared to other activation functions by preventing gradient oversaturation. Comparatively, the Tanh activation function gave the second-highest accuracy for these two datasets, from balancing gradient updates more than the Sigmoid activation function; although both of these yield substantially poorer results. Interestingly, these results were reversed for directional errors in the networks, returning the best results for the Sigmoid activation, and the worst results for ReLU activation; demonstrating how each activation function has unique advantages. Comparing 2nd, 3rd, and 4th-order polynomial regression shows how increasing regression order allows the algorithms to capture finer detail in fits, at the expense of overfitting and unstable behaviour. For the YFinance and Temperature datasets alike, lower-order linear regression gave higher accuracies due to this lack of overfitting - even outperforming optimised neural networks in one case. This is entirely reversed for MNIST image categorisation where neural networks obtained vastly superior results compared to linear regression. Based off our analysis of optimising machine-learning algorithms to individual datasets, future use of neural networks and regression algorithms on numerical and image processing can be greatly improved. Additionally, through the development of a custom desktop application to test these algorithms, we provide a valuable educational tool for future work in this field - boosting development in a thriving economic and research sector.

References

[1] McCulloch WS, Pitts W (December 1943). "A logical calculus of the ideas immanent in nervous activity". The Bulletin of Mathematical Biophysics. 5 (4): 115–133. Retrieved 1st March 2026

[2] Hebb D (2005) [1st Pub. 1949]. The Organization of Behavior. Taylor & Francis. ISBN 978-1-135-63190-1.

[3] Farley B, W.A. Clark (1954). "Simulation of Self-Organizing Systems by Digital Computer". IRE Transactions on Information Theory. 4 (4): 76–84.

[4] Rochester N, J.H. Holland, L.H. Habit, W.L. Duda (1956). "Tests on a cell assembly theory of the action of the brain, using a large digital computer". IRE Transactions on Information Theory. 2 (3): 80–93.

[5] Rosenblatt F (1958). "The Perceptron: A Probabilistic Model For Information Storage And Organization in the Brain". Psychological Review. 65 (6): 386–408.

[6] Rosenblatt F (1962). Principles of Neurodynamics. Spartan, New York.

[7] A. Ivakhnenko, V. G. Lapa (1966) Cybernetic Predicting Devices

[8] Robbins H, Monro S (1951). "A Stochastic Approximation Method". The Annals of Mathematical Statistics. 22 (3): 400

[9] Fukushima K (1979). "Neural network model for a mechanism of pattern recognition unaffected by shift in position—Neocognitron". Trans. IECE (In Japanese). J62-A (10): 658–665.

[10] Ciresan DC, Meier U, Masci J, Gambardella L, Schmidhuber J (2011). "Flexible, High Performance Convolutional Neural Networks for Image Classification". International Joint Conference on Artificial Intelligence.

[11] Gaudiaut T (25 November 2025). "The AI Market Is Poised for Explosive Growth". Statista. Retrieved 12th March 2026.

[12] Burkov, A. (2019). The Hundred-page machine learning book.

[13] Deng, L., 2012. The mnist database of handwritten digit images for machine learning research. IEEE Signal Processing Magazine, 29(6), pp.141–142.

[14] Xiao, H.; Rasul, K. & Vollgraf, R. (2017), 'Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms' , cite arxiv:1708.07747

[15] Bishop CM (2006). Pattern Recognition and Machine Learning. Springer.

[16] Hastie T, Tibshirani R, Friedman J (2009). The Elements of Statistical Learning: Data Mining, Inference, and Prediction. 2nd ed. Springer.

[17] Szandała, T (2020) Review and Comparison of Commonly Used Activation Functions for Deep Neural Networks , cite arXiv:2010.09458

[18] Hyndman RJ, Athanasopoulos G (2021). Forecasting: Principles and Practice. 3rd ed. OTexts.

[19] Cont R (2001). "Empirical properties of asset returns: stylized facts and statistical issues". Quantitative Finance. 1 (2): 223–236.

[20] Boyin Huang , Xungang Yin , Matthew J. Menne , et al. (2022). "Improvements to the Land Surface Air Temperature Reconstruction in NOAAGlobalTemp: An Artificial Neural Network Approach". Artificial Intelligence for the Earth Systems. Vol 1 Issue 4

[21] Rohde, R. A. and Hausfather, Z. (2020) "The Berkeley Earth Land/Ocean Temperature Record". Earth System Science Data. Vol. 12, Issue 4

[22] C. P. Morice, J. J. Kennedy, N. A. Rayner, et al. (2020) "An Updated Assessment of Near-Surface Temperature Change From 1850: The HadCRUT5 Data Set". JGR Atmospheres Volume 126, Issue 3

The Fashion MNIST dataset is freely available at https://github.com/zalandoresearch/fashion-mnist Benchmark is available at http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/.

Code used for each section of the project is freely available on John Ray's GitHub: https://github.com/john-ray-uk/PHYS379-Machine-Learning